



STUDENT

0002-CZD

TENTAMEN

250818 DV2606 2110

Salstentamen (DO)

Kurskod	--
Bedömningsform	--
Starttid	18.08.2025 09:00
Sluttid	18.08.2025 14:00
Bedömningsfrist	--
PDF skapad	05.11.2025 17:22
Skapad av	Anders Göransson

1.1 The cpu execution time can be described as $T_{exe} = I * CPI * T_c$. Which of the the following alternatives lower the CPI, thus increasing the performance?

☐ Increase the instruction-level parallelism.



☐ All alternatives are correct.



☐ Reduce the cache miss latency.

☐ Increase the cache hit ratio.

☐ Divide the instruction execution into more and smaller pipeline stages.

Fel. 0 av 1 poäng.

1.2 When we parallelize a sequential program, we find that we only can parallelize 95% of the program, i.e., 5% of the sequential execution time cannot be executed in parallel. What is true about the parallel program? (assuming the same input size, etc.)

☐ The speedup will be super-linear

☐ The parallel program can run at most 20 times faster than the sequential program.



☐ None of the alternatives is correct.

☐ The parallel program will be 95% faster than the sequential program.

☐ The parallel program can only run on a shared-memory multiprocessor.

Rätt. 1 av 1 poäng.

1.3 What is the (asymptotic) parallel time complexity of *matrix-matrix multiplication* using n^2 processors, assuming $n \times n$ matrices?

☐ $O(n)$



☒ $O(n^2)$



☐ None of the alternatives is correct.

☐ $O(n \log n)$

☐ $O(\log n)$

Fel. 0 av 1 poäng.

1.4 Assume a parallel application that is running on a parallel system with a certain efficiency (processor utilization). Which of the following alternatives would *decrease* the efficiency?

☒ All alternatives decrease the efficiency.



☐ Put more barrier synchronizations in the code.

☐ Increase the clock frequency of the processors.

☐ Increase the number of processors.

☐ Decrease the input data set size.

Rätt. 1 av 1 poäng.

1.5 Assume a parallel application that is running on a parallel system with a certain efficiency (processor utilization). Which of the following alternatives would *increase* the efficiency?

- ☐ None of the alternatives increases the efficiency.
- ☐ Increase the number of processors.
- ☐ Decrease the input data set size.
- ☐ Put more barrier synchronizations in the code.

☒ Increase the input data set size.



Rätt. 1 av 1 poäng.

1.6 Simultaneous multithreading is ...

☒ a technique to exploit parallelism in the instruction execution pipeline



- ☐ none of the alternatives is correct
- ☐ a software approach to parallelize an application
- ☐ a technique to improve branch prediction
- ☐ a technique to implement multi-level caches

Rätt. 1 av 1 poäng.

1.7 MSI (Modified-Shared-Invalid) is a basic write-invalidate protocol. Assuming that a particular cache block is in state Modified, which action is performed when the cache sees a read request on the shared bus?

- ☐ The processor is always stalled until the bus request is handled.
- ☒ The cache replies with the requested block, and the block is marked as Shared. 
- ☐ The cache replies with the requested block, and the block stays in state Modified.
- ☐ The request is denied since the block is in state Modified.
- ☐ None of the alternatives is correct.

Rätt. 1 av 1 poäng.

1.8 A fundamental difference between a directory-based cache coherence protocol and a snoopy-based cache coherence protocol is ...

- ☐ A directory-based protocol can only be implemented using the write-invalidate policy.
- ☐ A directory-based protocol relies on a shared broadcast medium.
- ☐ All of the alternatives are correct.
- ☐ A directory-based protocol can only be implemented using the write-update policy.
- ☒ A directory-based protocol keeps exact information in which caches copies of each memory block reside. 

Rätt. 1 av 1 poäng.

1.9 Which alternative is correct about memory consistency models?

- ☐ A memory consistency model is a specification of how the hardware handles the ordering of read and write operations. 
- ☒ A memory consistency model always guarantees a unique ordering of read and write operations. 
- ☐ None of the alternatives is correct.
- ☐ A memory consistency model always guarantees that the last written value to a variable is returned on a read operation.
- ☐ A memory consistency model always relies on specific synchronization operations.

Fel. 0 av 1 poäng.

1.10 In a MIMD parallel computer, the different processors (or processor nodes) are connected through an *interconnection network*. Which of the following metric is used when evaluating the performance of an interconnection network?

- ☐ CPU clock frequency
- ☐ None of the mentioned metrics is valid evaluation metrics
- ☒ Diameter 
- ☐ Memory capacity
- ☐ Disk read bandwidth

Rätt. 1 av 1 poäng.

1.11 How many neighbors does each internal node have in a 2-dimensional mesh network with $n \times n$ nodes, i.e., in total n^2 nodes?

☐ $\log n$

☒ 4



☐ n

☐ n^2

☐ $n \log n$

Rätt. 1 av 1 poäng.

1.12 A PRAM (Parallel Random Access Machine) can be useful for ...

☐ Running message-passing applications with GPU support.

☐ All of the alternatives are correct.

☐ Running applications with very large input data sets.

☐ Very easy implementation of parallel hardware.

☒ Designing and evaluating parallel algorithms.



Rätt. 1 av 1 poäng.

- 2.1** Shared-memory multiprocessors exist in two different variants: UMA (Uniform Memory Access) and NUMA (Non-uniform Memory Access) multiprocessors. Describe the primary difference between them, and also give an example when one is preferable over the other.

UMA it means when memory access is to be of uniformed time, it means that P1 memory access time should be same as P2 access time, this is seen in symmetric processor systems

NUMA is seen in distributed memory systems and where is the memory access not to be of uniformed time, the access not need to be same time because of latency and network.

example: if you building distributed system with computing horse power with multiple nodes that communicate over a network then NUMA is the best to go with. If you building physical compact computer then its best to go with UMA.

Ord: 104

Besvarad.

- 2.2** Explain why we need to *protect shared data* in shared memory systems.
Give an example what could happen if we do not protect shared data.
Discuss some common ways to protect shared data in shared memory systems.

why we need to protect shared data: in shared memory multiple thread/processor access the same data which can lead to race condition.

example suppose we 2 thread as increment of shared counter, when both thread reads the same value at same time before either writes it back, it may increase the counter by 1 instead of 2 which leads to losing updates.

to solve this: we can use mutexes/locks to allow one thread or process to access the critical section. We can also use semaphores to counter resource accesses by maintaining counter of available slots. Also we can use atomic operation using CPU instruction to execute or operate with one single uninterruptible step.

Ord: 113

Besvarad.

- 2.3** *Cache coherence* is an important issue to handle in a shared-memory multiprocessor. Describe, by giving an example, why ensuring cache coherence is important. (1p)
 Two main approaches exist to maintain cache coherence: *write-invalidate* and *write-update*. Briefly describe each of them, e.g., by drawing a simple picture. (2p)
 Describe some advantage and disadvantage with each of the approaches. (1p)

cache coherence is when the caches have same (up to date) data, if processor do operation on data of a cache, the modified data fetch directly to all other caches, this is important so the processors yield correct result and the one who check which cache need to be updated is handled by something called the snooper.

write update: it means the modified data of the cash should be reflected direct to all other caches, other processors get notified that the data is dirty and the new data fetched in. this is good because no need to wait for the modified data to come from different processor, and when the data is on the global memory it will be fetched directly.

write invalidate: means when processor do operation on data on cache this data stays invalidated and the other caches cannot execute any operation on it until the cache of the modified data allow them or give them access

disadvantage of write update is many global memory access which cause slowdown, advantage is it eliminate the need of complex implementation such as cache coherence which in write invalidate leads to MSI(modify, shared, invalid) protocol which is disadvantage of write invalidate, advantage of write invalidate is no many global memory access needed

Ord: 211

Besvarad.

- 2.4** When we parallelize a sequential application, e.g., by using threads or message-passing, so it can execute in parallel on N processors, we often find that the parallel program is not N times faster. Describe two important reasons for that.

first reason is Overhead of parallelism: extra work needed to create thread, manage them, synchronize them and handle the communication between them and this affect the speed.

second reason is the Amdhah's law: the porvition of the program can not be paralleld,even with many threads the sequential part takes time and the program need to wait this part to be done and this affect the speed.

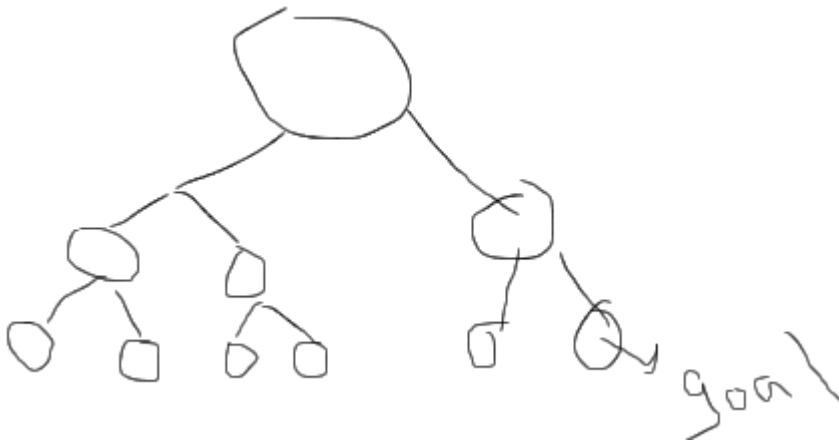
Ord: 66

Besvarad.

- 2.5** Sometimes we find that a system or application has a *superlinear speedup*. Clearly describe two situations when and why it may occur.

superliner speedup means when the algorithm / program runs faster than the sum of the individual processors which increase the execution time and performance.

first situation is lets say we want to do depth first research on tree and the goal is on the right sub tree. in the normal case we need to traverse throw all nodes to reach our goal. but we can use 2 processors one work with the left branch of the tree and the second processor work with the right branch of the tree and this way we find our goal faster



second situation: is lets say we want to do cache utilization. lets assume we have array with 1000 elements and cash memory max size is 200. we can use 5 processors and each have 200 size in this way we can perform cache utilization easy and fast.

Ord: 144

Besvarad.

3.1 Reduction sum

Consider the two functions SumReduction1 and SumReduction2 below:

```
__global__ void SumReduction1(float* input) {
    int i = 2*threadIdx.x
    for (int stride = 1; stride <= blockDim.x; stride *= 2) {
        if (threadIdx.x % stride == 0) {
            input[i] += input[i + stride];
        }
        __syncthreads();
    }
    // Save result
}

__global__ void SumReduction2(float* input) {
    int i = threadIdx.x
    for (int stride = blockDim.x; stride >= 1; stride /= 2) {
        if (threadIdx.x < stride) {
            input[i] += input[i + stride];
        }
        __syncthreads();
    }
    // Save result
}
```

Select one or more alternatives. Selecting an alternative that is incorrect will give a negative score. The minimum score on this assignment is 0.

☒ SumReduction1 uses a gather approach and SumReduction2 uses a scatter approach. 

☐ SumReduction1 has better memory coalescing (i.e., can benefit more from memory bursts) than SumReduction2.

☐ SumReduction1 has less control divergence than SumReduction2.

☒ SumReduction2 has better memory coalescing (i.e., can benefit more from memory bursts) than SumReduction1. 

☒ SumReduction2 has less control divergence than SumReduction1 

Delvis rätt. 1 av 2 poäng.

3.2 GPU and CUDA general

Select one or more alternatives. Selecting an alternative that is incorrect will give a negative score. The minimum score on this assignment is 0.

- ☐ The maximum number of threads that a GPU can handle is the same as the number of cores on the GPU.
- ☐ In general, the part of the silicon area that is used for implementing processing (e.g. ALUs, logic processing, addition, multiplication,...) is larger on a GPU compared to a CPU. ✓
- ☐ Four-dimensional data structures (and higher dimensions) cannot be processed on a GPU.

☒ A CUDA program may in some cases run exclusively on the GPU with no part of the program running on the CPU. ✗

☒ A CUDA program is a SPMD (Single Program Multiple Data) program. ✓

Delvis rätt. 0 av 2 poäng.

3.3 Thread coarsening

Select one or more alternatives. Selecting an alternative that is incorrect will give a negative score. The minimum score on this assignment is 0.

☐ Thread coarsening increases the parallelism in a CUDA program

☒ Thread coarsening can reduce the number redundant calculations in a CUDA program ✓

☐ Thread coarsening makes it possible to perform data transfer between the CPU and GPU in parallel with kernel execution on the GPU

☒ Thread coarsening can make it possible to use the shared memory more efficiently in a CUDA program ✓

☐ Thread coarsening decreases the completion time of a thread

Rätt. 2 av 2 poäng.

3.4

Select one or more alternatives. Selecting an alternative that is incorrect will give a negative score. The minimum score on this assignment is 0.

- ☐ The CPU can access variables in GPU shared memory
- ☐ Accesses to GPU global memory are faster than accesses to GPU shared memory.
- ☒ The content of the CUDA global memory is not changed between two kernel launches in the same program 
- ☐ All the threads in a thread block see the same registers
- ☒ Threads in different thread blocks cannot access the same variable in GPU shared memory 

Rätt. 2 av 2 poäng.

3.5 Thread Blocks and Warps

Select one or more alternatives. Selecting an alternative that is incorrect will give a negative score. The minimum score on this assignment is 0.

- ☐ A Thread Block cannot be larger than a Warp.
- ☒ Threads in the same Warp cannot execute different instructions at the same time. 
- ☐ Threads in the same Warp can execute on different Streaming Multiprocessors.
- ☐ Threads from different Thread Blocks can execute in the same Warp.
- ☒ A Thread Block can contain many Warps. 

Rätt. 2 av 2 poäng.

3.6

Consider the following small program.

```
for (int m = 0; m < M; m++) {  
    input[m].value += 5;  
    for (int n = 0; n < N; n++) {  
        output[n].value += input[m].value / distance(input[m].position, output[n].position);  
    }  
}
```

If we transform this program into a parallel CUDA program, we may run into some serious performance problems in the CUDA program.

What kind of performance problems may the CUDA program suffer from? How could we modify the small program above so that it is still functionally equivalent and so that the corresponding CUDA program does not suffer from any serious performance problems?

global memory access issue: many threads might write to the same output[n].value

non coalesced memory access: the irregular access of input[m].value

the distance is repeatedly/repeated calculated for the same pairs

the nested for loop may not map correctly and lead to underutilization of GPU resources

for solve this issues:

assign each thread to a unique (unik) input[m] output[n]

calculate the distance once and store it in shared memory

use atomic operation for better GPU resource utilization

Ord: 76

Besvarad.

3.7

Explain how sparse matrices are stored when using the ELL format. Also explain the potential performance advantages for a CUDA program when using the ELL format compared to the CSR (compressed sparse row) format. Also, explain how sparse matrices are stored when using the COO (coordinate list) format. Also, explain why we sometimes would like to combine the ELL and COO formats when using CUDA. Give an example of how the ELL and COO formats can be combined when storing a sparse matrix.

ELL format: matrix is stored in two 2D arrays values and columns. Rows are padded with nonzeros so each row have same length

CUDA vs CSR:

regular memory layout the rows have same number of elements

colasced memory reads

CSR required variety of length, more indirection

COO format: matrix stored in three different arrays the equal length values, rows, columns. its flexible and simple, good for construct/ modifying matrices/ matrix parallel/ program application.

ELL and COO:

ELL used most for the regular part of the matrix when padding nonzeros so row get same length

COO is to store extra elements from large rows to avoid large padding

example: rows less/equal to 5 -> ELL

store extra nonzeros -> COO

Ord: 119

Besvarad.

3.8

How can the programmer make the GPU put a variable in the cache? Where is such a variable stored when it is not in the cache?

its by constructing the program to use GPU only instead of CPU, when it not in the cache its stored in the global memory or memory bus

Ord: 27

Besvarad.

3.9 Privatization

Privatization is a technique that can be used for improving the performance of some CUDA programs, e.g., programs that perform parallel histogramming. Explain how privatization works and why privatization can improve performance.

Privatization is when multiple threads access the same memory location at the same time and that can lead for race condition and inconsistency in the memory.

We can prevent this by using atomic operation, atomic operation decrease the execution time through idle time and latency, but we can assign for each thread "private copy" that can work on it and then in the end threads merge to present the result and this technique is called Privatization. so no race condition occur

Ord: 81

Besvarad.

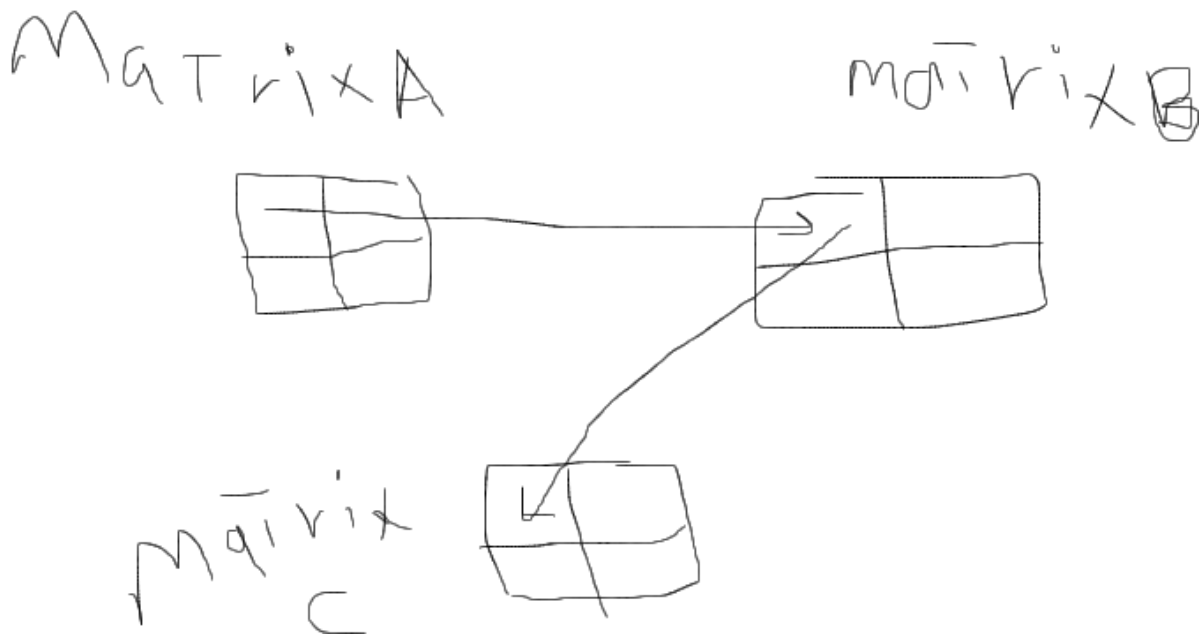
3.10 Tiling

Tiling is a technique that can be used for improving the performance of some CUDA programs, e.g., programs that perform parallel matrix multiplication or convolution. Explain how tiling works, why it can increase performance and why and how we may need to use `__syncthreads()` when using tiling.

tiling is when dividing the data into smaller tiles, each thread load these tile computing result locally and then writes it back to the global memory.

tiling reduce global memory access use shared memory because its faster and more coalesced. tiling enable efficient global memory access use shared memory efficiency. use `__syncthreads()` is used for so all threads finishes loading thier tiles before program computation begin

example; matrix tiles of data loaded into shared memory and we use `__syncthreads()` for the same purpose mention before



Ord: 84

Besvarad.